

Figure 1: Entity tracking results for DeepSeek, Gemma, and Llama 2 models. Error bars indicate 95% confidence intervals, and the black dashed lines show the performance of the random baseline.

We noticed that smaller models suffered from formatting issues, often deviating from the format specified by the prompt or omitting the contents of some boxes. For this reason, we used regular expression-based constrained decoding using the `outlines` library (Willard & Louf, 2023).²

We report all results divided into the number of operations affecting the target box rather than reporting one aggregate accuracy metric. This is to distinguish trivial cases from cases that actually require tracking state changes—when the number of operations affecting the target box is 0, simply copying from the initial state description yields the correct answer. Furthermore, we compare the model results to the strong random baseline by Kim & Schuster (2023). For this baseline, we randomly sample 0 to 3 objects for each box from the set of objects that have been previously mentioned in a clause with the box in question.

3.3 Experiment 1: Effect of Code

Figure 1 compares the entity tracking performance of base models (red lines) and code models (blue lines) for models from the DeepSeek (DeepSeek-AI et al., 2024; Guo et al., 2024), Gemma (Gemma Team et al., 2024), and Llama 2 families of various sizes (Touvron et al., 2023b; Rozière et al., 2024). In general, we find clear evidence that continued training on large amounts of code improves entity tracking abilities, as can be seen for the Llama 2 13B and 70B models as well as for the DeepSeek models. In these model comparisons, the models trained on code consistently outperformed the base models on the nontrivial cases of entity tracking (number of operations affecting box state ≥ 1). In the case of 13B models, a boost in trivial cases is also observed (number of operations = 0); in 70B models, performance on the trivial cases is already saturated in the base model.

In Llama 2 7B models, the gains through additional code training are relatively minor, with most of the gains deriving from boosts in examples where the number of operations is either 0 or 1. Similarly minor gains were observed in CodeGemma 8B, except that the gains were observed in examples with 1 and 2 operations. For both of these models, we also observed that for number of operations greater than 0 (Llama 2 7B) and 2 (Gemma 8B), neither the base nor the code variants perform better than our random baseline. These results suggest that there is both a possible effect of scale in the effectiveness of code training as observed in the Llama 2 series, and an effect of the amount of additional code training (DeepSeek-Coder: 2T tokens, Code Llama: 500B tokens).

3.4 Experiment 2: Effect of Math

In evaluating the effect of additional math training, we start by revisiting the claim of Prakash et al. (2024) that the FLoat model obtained by finetuning Llama 7B (Touvron et al., 2023a) on arithmetic tasks from Liu & Low (2023) yields superior entity tracking

²<https://github.com/outlines-dev/outlines>

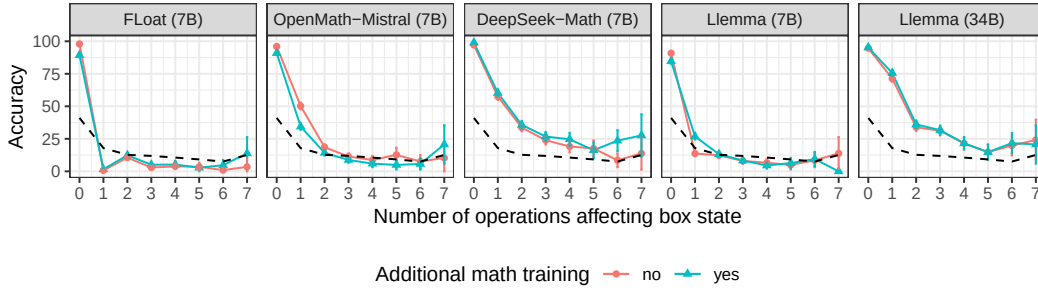


Figure 2: Entity tracking results for models trained with additional math data. See Table 1 for the model names of the base and math models. Error bars indicate 95% confidence intervals, and the black dashed lines show the performance of the random baseline.

performance. As can be seen in Table 3, FLoat did show slightly higher accuracy on the non-trivial tracking cases (number of operations ≥ 1) than the base model, but the gain was marginal.³ Furthermore, neither the base Llama model nor the FLoat model performed better than our random baseline on non-trivial entity tracking examples (Figure 2, far left).

Model	Aggregate	NumOps = 0	NumOps ≥ 1
Llama 7B	28.67	97.93	4.34
FLoat 7B	27.55	89.33	5.85

Table 3: Llama 7B vs. FLoat 7B results.

Following this observation, we compared Mistral and OpenMathMistral models where the latter is a model further trained on OpenMathInstruct-1, a synthetically generated instruction-tuning dataset containing 1.8M unique solutions to math problems sourced from MATH and GSM8K datasets (Toshniwal et al., 2024). As shown in Figure 2, OpenMathMistral only achieved marginal gains over the base Mistral model when there are 7 operations affecting the target box, and in most other cases, the base model consistently outperforms the math-finetuned model. Furthermore, neither model outperformed the random baseline for examples with more than 2 operations affecting the box of interest.

The unclear benefit of additional math training is further corroborated by marginal gains in models trained on math data that are not in “instruct” format like FLoat and OpenMathMistral. Figure 2 shows that DeepSeek-Math (Shao et al., 2024) performed close to the DeepSeek-Coder model for most cases. The gains are even more limited in the comparison between Code Llama vs Llemma (Azerbayev et al., 2024). Llemma 34B outperformed Code Llama 34B by a narrow margin for the non-trivial tracking cases (Llemma: 47.86, Code Llama: 45.46 for examples where the number of operations ≥ 1). These results suggest a limited benefit of additional math pretraining on entity tracking.

3.5 Experiment 3: Effect of Alignment Tuning

Finally, we explore the effect of alignment tuning on entity tracking. For models in the Llama 2 family, alignment tuning the base models led to minor gains (Figure 3, orange vs. green lines in the top row panels), whereas alignment-tuned code models did not consistently lead to gains and sometimes performed worse than the non-alignment-tuned counterparts (orange vs. green lines in the bottom row panels). Nevertheless, the best-performing model was CodeLlama 70B-instruct (64.9 accuracy on 1+ operations), combining code and alignment tuning.

³These numbers are not expected to align with numbers reported in Prakash et al. (2024) because they used a modified version of the original task.

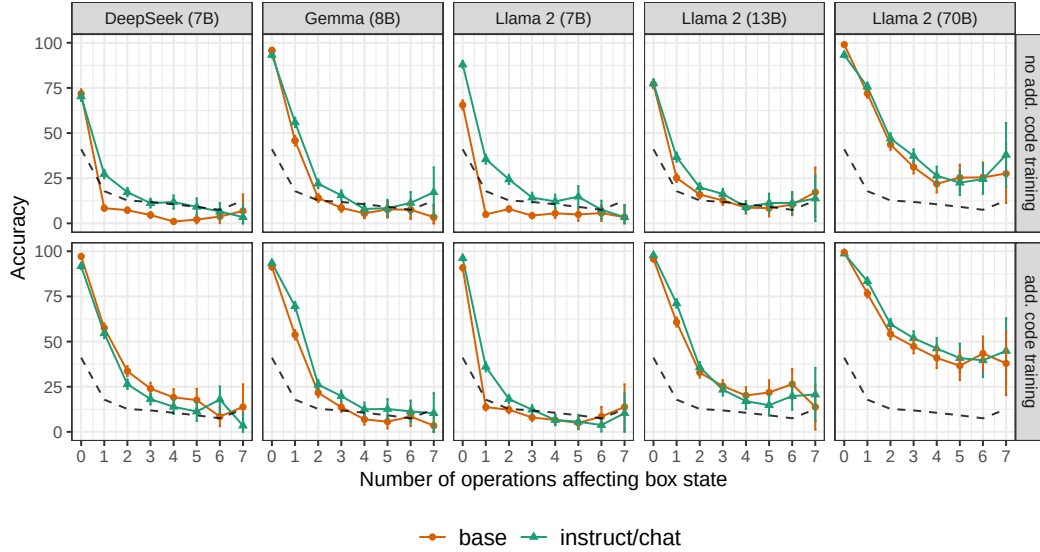


Figure 3: Entity tracking results for alignment-tuned DeepSeek, Gemma and Llama 2 models. The top panels show models without additional code training, whereas the bottom panels show models that have been trained on additional amounts of code before alignment tuning. Error bars indicate 95% confidence intervals, and the black dashed lines show the performance of the random baseline.

The DeepSeek models showed similar trends to the general observation made above: alignment tuning of the base model led to gains, whereas alignment tuning of the code model did not. Gemma 8B and CodeGemma 8B models did benefit from alignment tuning, similarly to Llama 2 7B and CodeLlama 7B models, although the gains were smaller.

Overall, alignment tuning affects base and code models differently, where the gains for base models tend to be greater. The benefit of alignment tuning for base models seems to be inversely correlated with scale: smaller base models benefit more from alignment tuning.

4 Conclusion and Future Work

We explored the effect of code, math, and alignment tuning on LLMs’ capacity to track entities in natural language text. Our main findings are threefold:

1. Additional code training leads to consistent improvements across model families and sizes.
2. Additional math training does not yield consistent improvements, and the performance gains are at best marginal.
3. Alignment tuning leads to different patterns of improvement depending on whether it was applied to base models or code models. Base models consistently benefit from alignment tuning, and smaller models see more improvement. The benefit for code models is more mixed, but the best performance is achieved through combining code and instruction tuning.

Our work thus adds to a growing body of literature that suggests that pretraining on code improves LLM performance on reasoning tasks, including commonsense reasoning (Madaan et al., 2022), chain-of-thought reasoning (Wei et al., 2022b), mathematical problems (Razeghi et al., 2024), and entity tracking tasks (Muennighoff et al., 2023). *Why might this be the case?* Kim & Schuster (2023) argued that keeping track of the states of variables is important for producing correct code, and hypothesized that this kind of procedural input may provide a